SHEKH EHSANUR RAHMAN

Senior Flutter Engineer | Native iOS · Android · macOS

Offline-first sync · Payments & subscriptions · Custom plugins & native integration · Release automation

mail@ehsanur.com · +880 1747-931646 (WhatsApp) · ehsanur.com · linkedin.com/in/sujit70777 · github.com/sujit70777

Remote from Bangladesh (UTC+6) · **No relocation or visa sponsorship required** · Full-time or contract · Works with any time zone; flexible core overlap hours · Available now

~0.5 s native scanner per QR code or barcode	**~1 month** to launch an app stalled for ~2 years	**10–20 min** release time, down from ~1 hour	**2 × 100K+** downloads, offline-first learning apps

WHO I AM

Senior Flutter Engineer with 11 years of shipping mobile apps (7 of them in Flutter) — 50+ apps worked on, 1M+ combined users — and the native depth to go below the framework: platform channels and plugins in Swift, Kotlin and Java, home-screen widgets, and a native macOS app of my own. I take on the problems that block a product — offline data that must not be lost, payments that must reconcile across Apple and Stripe, platform edge cases, and slow components that need a native rewrite. As mobile team lead I hired and mentored developers, ran sprints, owned releases and worked directly with product and design. Two products of my own live on the Mac App Store and Google Play.

SELECTED WORK

Native receipt scanner — expense-reimbursement FinTech · *Flutter · Swift · Java · WebSockets*

- **Problem:** the generic Flutter scanner was too slow and couldn't be customized; users waited on an endless spinner during verification.
- **Decision:** rewrote scanning natively behind a Flutter interface for direct control of camera handling; live verification progress over WebSockets; offline claims that sync later.
- **Result:** ~0.5 s per code; a reimbursement completes ~5–10 s after the scan. Open-sourced as a Flutter package.

One payment model for Stripe and Apple — subscription app · *Flutter · StoreKit 2 · Stripe · backend*

- **Problem:** Stripe on Android and Apple In-App Purchase on iOS disagreed — duplicate purchases, mismatched access; the app had been stuck ~2 years.
- **Decision:** as team lead, made one account-level entitlement on the backend the single source of truth for both payment systems, with duplicate-purchase checks and record reconciliation.
- **Result:** approved and live on the App Store ~1 month after stabilization began.

Offline-first learning apps — EdTech · *Flutter · Firebase · SQLite*

- **Problem:** students on weak internet lost lessons and progress whenever the network dropped.
- **Decision:** led a 6-developer team; device as the source of truth with a Firebase/SQLite sync layer and deterministic conflict resolution.
- **Result:** two of the apps each passed 100,000 downloads on Google Play.

ALSO

Hourwise — native macOS app I built solo, live on the Mac App Store, with on-device machine learning that keeps user data on the Mac (SwiftUI, AppKit, ~17k lines of Swift) · **Repha** — B2B ordering network for pharmacies and medical reps I built solo, live on Google Play: 70,000+ medicines searchable offline, one list auto-split by company, photo-to-order OCR, Laravel backend at zero hosting cost · **9 Flutter packages** on pub.dev, including a CRDT offline-sync layer and a high-latitude prayer-time engine · **Releases** automated with Fastlane + GitHub Actions · **Team lead** of 6 developers; trained 200+ developers.

CORE STACK

Flutter · Dart · BLoC · Riverpod · Clean Architecture · platform channels · Swift · SwiftUI · AppKit · Kotlin · Java · Firebase · SQLite (Drift, GRDB) · offline-first sync · Supabase · Laravel + MySQL · StoreKit 2 · Stripe · Fastlane · GitHub Actions · Sentry · Crashlytics